

Abstract Data Types and Encapsulation Constructs

- 11.1** The Concept of Abstraction
- 11.2** Introduction to Data Abstraction
- 11.3** Design Issues for Abstract Data Types
- 11.4** Language Examples
- 11.5** Parameterized Abstract Data Types
- 11.6** Encapsulation Constructs
- 11.7** Naming Encapsulations

In this chapter, we explore programming language constructs that support data abstraction. Among the new ideas of the last 50 years in programming methodologies and programming language design, data abstraction is one of the most profound.

We begin by discussing the general concept of abstraction in programming and programming languages. Data abstraction is then defined and illustrated with an example. This topic is followed by descriptions of the support for data abstraction in C++, Java, C#, and Ruby. To illuminate the similarities and differences in the design of the language facilities that support data abstraction, implementations of the same example data abstraction are given in C++, Java, and Ruby. Next, the capabilities of C++, Java 5.0, and C# 2005 to build parameterized abstract data types are discussed.

All the languages used in this chapter to illustrate the concepts and constructs of abstract data types support object-oriented programming, because virtually all contemporary languages support object-oriented programming and nearly all of those that do not, and yet support abstract data types, have faded into obscurity.

Constructs that support abstract data types are encapsulations of the data and operations on objects of the type. Encapsulations that contain multiple types are required for the construction of larger programs. These encapsulations and the associated namespace issues are also discussed in this chapter.

Some programming languages support logical, as opposed to physical, encapsulations, which are actually used to encapsulate names. These are discussed in Section 11.7.

11.1 The Concept of Abstraction

An **abstraction** is a view or representation of an entity that includes only the most significant attributes. In a general sense, abstraction allows one to collect instances of entities into groups in which their common attributes need not be considered. For example, suppose we define birds to be creatures with the following attributes: two wings, two legs, a tail, and feathers. Then, if we say a crow is a bird, a description of a crow need not include those attributes. The same is true for robins, sparrows, and yellow-bellied sapsuckers. The common attributes in the descriptions of specific species of birds can be abstracted away, because all species have them. Within a particular species, only the attributes that distinguish that species need be considered. For example, crows have the attributes of being black, being of a particular size, and being noisy. A description of a crow needs to provide those attributes, but not the others that are common to all birds. This results in significant simplification of the descriptions of members of the species. A less abstract view of a species, that of a bird, may be considered when it is necessary to see a higher level of detail, rather than just the special attributes.

In the world of programming languages, abstraction is a weapon against the complexity of programming; its purpose is to simplify the programming process. It is an effective weapon because it allows programmers to focus on essential attributes, while ignoring subordinate attributes.

The two fundamental kinds of abstraction in contemporary programming languages are process abstraction and data abstraction.

The concept of **process abstraction** is among the oldest in programming language design (Plankalkül supported process abstraction in the 1940s). All subprograms are process abstractions because they provide a way for a program to specify a process, without providing the details of how it performs its task (at least in the calling program). For example, when a program needs to sort an array of numeric data of some type, it usually uses a subprogram for the sorting process. At the point where the sorting process is required, a statement such as

```
sortInt(list, listLen)
```

is placed in the program. This call is an abstraction of the actual sorting process, whose algorithm is not specified. The call is independent of the algorithm implemented in the called subprogram.

In the case of the subprogram `sortInt`, the only essential attributes are the name of the array to be sorted, the type of its elements, the array's length, and the fact that the call to `sortInt` will result in the array being sorted. The particular algorithm that `sortInt` implements is an attribute that is not essential to the user. The user needs to see only the name and protocol of the sorting subprogram to be able to use it.

The widespread use of data abstraction necessarily followed that of process abstraction because an integral and essential part of every data abstraction is its operations, which are defined as process abstractions.

11.2 Introduction to Data Abstraction

The evolution of data abstraction began in 1960 with the first version of COBOL, which included the record data structure.¹ The C-based languages have structs, which are also records. An abstract data type is a data structure, in the form of a record, but which includes subprograms that manipulate its data.

Syntactically, an abstract data type is an enclosure that includes only the data representation of one specific data type and the subprograms that provide the operations for that type. Through access controls, unnecessary details of the type can be hidden from units outside the enclosure that use the type. Program units that use an abstract data type can declare variables of that type, even though the actual representation is hidden from them. An instance of an abstract data type is called an **object**.

One of the motivations for data abstraction is similar to that of process abstraction. It is a weapon against complexity; a means of making large and/or complicated programs more manageable. Other motivations for and advantages of abstract data types are discussed later in this section.

1. Recall from Chapter 2 that a record is a data structure that stores fields, which have names and can be of different types.

Object-oriented programming, which is described in Chapter 12, is an outgrowth of the use of data abstraction in software development, and data abstraction is one of its fundamental components.

11.2.1 Floating-Point as an Abstract Data Type

The concept of an abstract data type, at least in terms of built-in types, is not a recent development. All built-in data types, even those of Fortran I, are abstract data types, although they are rarely called that. For example, consider a floating-point data type. Most programming languages include at least one of these. A floating-point type provides the means to create variables to store floating-point data and also provides a set of arithmetic operations for manipulating objects of the type.

Floating-point types in high-level languages employ a key concept in data abstraction: information hiding. The actual format of the floating-point data value in a memory cell is hidden from the user, and the only operations available are those provided by the language. The user is not allowed to create new operations on data of the type, except those that can be constructed using the built-in operations. The user cannot directly manipulate the parts of the actual representation of values because that representation is hidden. It is this feature that allows program portability between implementations of a particular language, even though the implementations may use different representations for particular data types. For example, before the IEEE 754 standard floating-point representations appeared in the mid-1980s, there were several different representations being used by different computer architectures. However, this variation did not prevent programs that used floating-point types from being portable among the various architectures.

11.2.2 User-Defined Abstract Data Types

A user-defined abstract data type should provide the same characteristics as those of language-defined types, such as a floating-point type: (1) a type definition that allows program units to declare variables of the type but hides the representation of objects of the type; and (2) a set of operations for manipulating objects of the type.

We now formally define an abstract data type in the context of user-defined types. An **abstract data type** is a data type that satisfies the following conditions:

- The representation of objects of the type is hidden from the program units that use the type, so the only direct operations possible on those objects are those provided in the type's definition.
- The declarations of the type and the protocols of the operations on objects of the type, which provide the type's interface, are contained in a single syntactic unit. The type's interface does not depend on the representation of the objects or the implementation of the operations. Also, other program units are allowed to create variables of the defined type.

There are several benefits of information hiding. One of these is increased reliability. Program units that use a specific abstract data type are called **clients** of that type. Clients cannot manipulate the underlying representations of

objects directly, either intentionally or by accident, thus increasing the integrity of such objects. Objects can be changed only through the provided operations.

Another benefit of information hiding is it reduces the range of code and number of variables of which a programmer must be aware when writing or reading a part of the program. The value of a particular variable can be changed only by code in a restricted range, making the code easier to understand and making it less challenging to find sources of incorrect changes.

Information hiding also makes name conflicts less likely, because the scopes of variables is smaller.

Finally, consider the following advantage of information hiding: Suppose that the original implementation of the stack abstraction uses a linked list representation. At a later time, because of memory management problems with that representation, the stack abstraction is changed to use a contiguous representation (one that implements a stack in an array). Because data abstraction was used, this change can be made in the code that defines the stack type, but no changes will be required in any of the clients of the stack abstraction. Of course, a change in protocol of any of the operations would require changes in the clients.

Although the definition of an abstract data type specifies that data members of objects must be hidden from clients, many situations arise in which clients need to access these data members. The common solution is to provide accessor methods, sometimes called **getters** and **setters**, that allow clients indirect access to the so-called hidden data—a better solution than simply making the data public, which would provide direct access. There are three reasons why accessors are better:

1. Read-only access can be provided by having a getter method but no corresponding setter method.
2. Constraints can be included in setters. For example, if the data value should be restricted to a particular range, the setter can enforce that.
3. The actual implementation of the data member can be changed without affecting the clients if getters and setters are the only access.

Both specifying data in an abstract data type to be public and providing accessor methods for that data are violations of the principles of abstract data types. Some believe these are simply loopholes that make an imperfect design usable. As we will see in Section 11.4.4.2, Ruby disallows making instance data public. However, Ruby also makes it very easy to create accessor functions. It is a challenge for developers to design abstract data types in which all of the data is actually hidden.

The primary advantage of packaging the declarations of the type and its operations in a single syntactic unit is that it provides a method of organizing a program into logical units that can be compiled separately. In some cases, the implementation is included with the type declaration; in other cases, it is in a separate syntactic unit. The advantage of having the implementation of the type and its operations in different syntactic units is that it increases the program's modularity and it is a clear separation of design and implementation. If both the declarations and the definitions of types and operations are in the same syntactic unit, there must be some means of hiding from client program units the parts of the unit that specify the definitions.

11.2.3 An Example

A stack is a widely applicable data structure that stores some number of data elements and only allows access to the data element at one of its ends, the top. Suppose an abstract data type is to be constructed for a stack that has the following abstract operations:

<code>create(stack)</code>	Creates and possibly initializes a stack object
<code>destroy(stack)</code>	Deallocates the storage for the stack
<code>empty(stack)</code>	A predicate (or Boolean) function that returns true if the specified stack is empty and false otherwise
<code>push(stack, element)</code>	Pushes the specified element on the specified stack
<code>pop(stack)</code>	Removes the top element from the specified stack
<code>top(stack)</code>	Returns a copy of the top element from the specified stack

Note that some implementations of abstract data types do not require the create and destroy operations. For example, simply defining a variable to be of an abstract data type may implicitly create the underlying data structure and initialize it. The storage for such a variable may be implicitly deallocated at the end of the variable's scope.

A client of the stack type could have a code sequence such as the following:

```
. . .
create(stk1);
push(stk1, color1);
push(stk1, color2);
temp = top(stk1);
. . .
```

11.3 Design Issues for Abstract Data Types

A facility for defining abstract data types in a language must provide a syntactic unit that encloses the declaration of the type and the prototypes of the subprograms that implement the operations on objects of the type. It must be possible to make these visible to clients of the abstraction. This allows clients to declare variables of the abstract type and manipulate their values. Although the type name must have external visibility, the type representation must be hidden. The type representation and the definitions of the subprograms that implement the operations may appear inside or outside this syntactic unit.

Few, if any, general built-in operations should be provided for objects of abstract data types, other than those provided with the type definition. There simply are not many operations that apply to a broad range of abstract data

types. Among these are assignment and comparisons for equality and inequality. If the language does not allow users to overload assignment, the assignment operation must be included in the abstraction. Comparisons for equality and inequality should be predefined in the abstraction in some cases but not in others. For example, if the type is implemented as a pointer, equality may mean pointer equality, but the designer may want it to mean equality of the structures referenced by the pointers.

Some operations are required by many abstract data types, but because they are not universal, they often must be provided by the designer of the type. Among these are iterators, accessors, constructors, and destructors. Iterators were discussed in Chapter 8. Accessors provide a form of access to data that is hidden from direct access by clients. Constructors are used to initialize parts of newly created objects. Destructors are often used to reclaim heap storage that may be used by parts of abstract data type objects in languages that do not do implicit storage reclamation.

As stated earlier, the enclosure for an abstract data type defines a single data type and its operations. Many contemporary languages, including C++, Java, and C#, directly support abstract data types.

The first design issue is whether abstract data types can be parameterized. For example, if the language supports parameterized abstract data types, one could design an abstract data type for some structure that could store elements of any type. Parameterized abstract data types are discussed in Section 11.5. The second design issue is what access controls are provided and how such controls are specified. Finally, the language designer must decide whether the specification of the type is physically separate from its implementation (or whether that is a developer choice).

11.4 Language Examples

The concept of data abstraction had its origins in SIMULA 67, although that language did not provide complete support for abstract data types, because it did not include a way to hide implementation details. In this section, we describe the support for data abstraction provided by C++, Java, C#, and Ruby.

11.4.1 Abstract Data Types in C++

C++, which was first released in 1985, was created by adding features to C. The first important additions were those to support object-oriented programming. Because one of the primary components of object-oriented programming is abstract data types, C++ obviously is required to support them.

C++ provides two constructs that are very similar to each other, the class and the struct, which directly support abstract data types. Because structs are most commonly used when only data is included, we do not discuss them further here.



C++: Its Birth, Its Ubiquitousness, and Common Criticisms

BJARNE STROUSTRUP

Bjarne Stroustrup is the designer and original implementer of C++ and the author of *A Tour of C++, Programming—Principles and Practice using C++, The C++ Programming Language, The Design and Evolution of C++*, and many other publications. His research interests include distributed systems, design, programming techniques, software development tools, and programming languages. He is actively involved in the ANSI/ISO standardization of C++. Dr. Stroustrup is a Managing Director in the technology division of Morgan Stanley in New York City, a Visiting Professor in Computer Science at Columbia University, and a Distinguished Research Professor in Computer Science at Texas A&M University. He is a member of the National Academy of Engineering, an ACM Fellow, and an IEEE fellow. In 1993, Stroustrup received the ACM Grace Murray Hopper Award “for his early work laying the foundations for the C++ programming language. Based on the foundations and Dr. Stroustrup’s continuing efforts, C++ has become one of the most influential programming languages in the history of computing.”

(year of interview: 2002)

A BRIEF HISTORY OF YOU AND COMPUTING

What were you working on, and where, before you joined Bell Labs in the early 1980s? At Bell Labs, I was doing research in the general area of distributed systems. I joined in 1979. Before that, I was finishing my Ph.D. in that field in Cambridge University.

Did you immediately start on “C with Classes” (which would later become C++)? I worked on a few projects related to distributed computing before starting on C with Classes and during the development of that and of C++. For example, I was trying to find a way to distribute the UNIX kernel across several computers and helped a lot of projects build simulators.

Was it an interest in mathematics that got you into this profession? I signed up for a degree in “mathematics with computer science” and my master’s degree is officially a math degree. I—wrongly—thought that computing was some kind of applied math. I did a couple of years of math and rate myself a poor mathematician, but that’s still much better than not knowing math. At the time I signed up, I had never even seen a computer. What I love about

computing is the programming rather than the more mathematical fields.

DISSECTING A SUCCESSFUL LANGUAGE

I’d like to work backward, listing some items I think make C++ ubiquitous, and get your reaction. It’s “open source,” nonproprietary, and standardized by ANSI/ISO. The ISO C++ standard is important. There are many independently developed and evolving C++ implementations. Without a standard for them to adhere to and a standards process to help coordinate the evolution of C++, a chaos of dialects would erupt.

It is also important that there are both open-source and commercial implementations available. In addition, for many users, it is crucial that the standard provides a measure of protection from manipulation by implementation providers.

The ISO standards process is open and democratic. The C++ committee rarely meets with fewer than 50 people present and typically more than eight nations are represented at each meeting. It is not just a vendors’ forum.

It's ideal for systems programming (which, at the time C++ was born, was the largest sector of the market developing code).

Yes, C++ is a strong contender for any systems-programming project. It is also effective for embedded systems programming, which is currently the fastest-growing sector. Yet another growth area for C++ is high-performance numeric/engineering/scientific programming.

Its object-oriented nature and inclusion of classes/libraries make programming more efficient and transparent. C++ is a multiparadigm programming language. That is, it supports several fundamental styles of programming (including object-oriented programming) and combinations of those styles. When used well, this leads to cleaner, more flexible, and more efficient libraries than can be provided using just one paradigm. The C++ standard library containers and algorithms, which is basically a generic programming framework, is an example. When used together with (object-oriented) class hierarchies, the result is an unsurpassed combination of type safety, efficiency, and flexibility.

Its incubation in the AT&T development environment AT&T Bell Labs provided an environment that was crucial for C++'s development. The labs were an exceptionally rich source of challenging problems and a uniquely supportive environment for practical research. C++ emerged from the same research lab as C did and benefited from the same intellectual tradition, experience, and exceptional people. Throughout, AT&T supported the standardization of C++. However, C++ was not the beneficiary of a massive marketing campaign, like many modern languages. That's simply not the way the labs work.

Did I miss anything on your top list? Undoubtedly.

Now, let me paraphrase from the C++ critiques and get your reactions: It's huge/unwieldy. The "hello world" problem is 10 times larger in C++ than in C. C++ is certainly not a small language, but then few modern languages are. If a language is small, you tend to need huge libraries to get work done and often

have to rely on conventions and extensions. I prefer to have key parts of the inevitable complexity in the language where it can be seen, taught, and effectively standardized rather than hidden elsewhere in a system. For most purposes, I don't consider C++ unwieldy. The C++ "hello world" program isn't larger than its C equivalent on my machine, and it shouldn't be on yours.

In fact, the object code for the C++ version of the "hello world" program is smaller than the C version on my machine. There is no language reason why the one version should be larger than the other. It is all an issue of how the implementor organized the libraries. If one version is significantly larger than the other, report the problem to the implementor of the larger version.

It's tougher to program in C++ (compared with C). (Something the critics say.) Even you once admitted it, saying something about shooting yourself in the foot with C versus C++. Yes, I did say something along the lines of "C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do, C++ blows your whole leg off." What people tend to miss is that what I said about C++ is to a varying extent true for all powerful languages. As you protect people from simple dangers, they get themselves into new and less obvious problems. Someone who avoids the simple problems may simply be heading for a not-so-simple one. One problem with very supporting and protective environments is that the hard problems may be discovered too late or be too hard to remedy once discovered. Also, a rare problem is harder to find than a frequent one because you don't suspect it.

It's appropriate for embedded systems of today but not for the Internet software of today. C++ is suitable for embedded systems today. It is also suitable—and widely used—for "Internet software" today. For example, have a look at my "C++ applications" Web page. You'll notice that some of the major Web service providers, such as Amazon, Adobe, Google, Quicken, and Microsoft, critically rely on C++. Gaming is a related area in which you find heavy C++ use.

Did I miss another one that you get a lot? Sure.

C++ classes are types. A C++ program unit that declares an instance of a class can also access any of the public entities in that class, but only through an instance of the class.

11.4.1.1 Encapsulation

The data defined in a C++ class are called **data members**; the functions (methods) defined in a class are called **member functions**. Data members and member functions appear in two categories: class and instance. Class members are associated with the class; instance members are associated with the instances of the class. In this chapter, only the instance members of a class are discussed. All of the instances of a class share a single set of member functions, but each instance has its own set of the class's data members. Class instances can be static, stack dynamic, or heap dynamic. If static or stack dynamic, they are referenced directly with value variables. If heap dynamic, they are referenced through pointers. Stack dynamic instances of classes are always created by the elaboration of an object declaration. Furthermore, the lifetime of such a class instance ends when the end of the scope of its declaration is reached. Heap dynamic class instances are created with the **new** operator and destroyed with the **delete** operator. Both stack- and heap-dynamic classes can have pointer data members that reference heap dynamic data, so that even though a class instance is stack dynamic, it can include data members that reference heap dynamic data.

A member function of a class can be defined in two distinct ways: The complete definition can appear in the class, or only in its header. When both the header and the body of a member function appear in the class definition, the member function is implicitly inlined. Recall that this means that its code is placed in the caller's code, rather than requiring the usual call and return linkage. If only the header of a member function appears in the class definition, its complete definition appears outside the class and is separately compiled. The rationale for allowing member functions to be inlined was to save function call overhead in real-time applications, in which run-time efficiency is of utmost importance. The downside of inlining member functions is that it clutters the class definition interface, resulting in a reduction in readability.

Placing member function definitions outside the class definition separates specification from implementation, a common goal of modern programming.

11.4.1.2 Information Hiding

A C++ class can contain both hidden and visible entities (meaning they are either hidden from or visible to clients of the class). Entities that are to be hidden are placed in a **private** clause, and visible, or public, entities appear in a **public** clause. The **public** clause therefore describes the interface to class instances. There is also a third category of visibility, **protected**, which makes a member visible to subclasses, but not to clients.

11.4.1.3 Constructors and Destructors

C++ allows the user to include **constructor** functions in class definitions, which are used to initialize the data members of newly created objects. A constructor may also allocate the heap-dynamic data that are referenced by the pointer members of the new object. Constructors are implicitly called when an object of the class type is created. A constructor has the same name as the class whose objects it initializes. Constructors can be overloaded, but of course each constructor of a class must have a unique parameter profile.

A C++ class can also include a function called a **destructor**, which is implicitly called when the lifetime of an instance of the class ends. As stated earlier, stack-dynamic class instances can contain pointer members that reference heap-dynamic data. The destructor function for such an instance can include a **delete** operator on the pointer members to deallocate the heap space they reference. Destructors are often used as a debugging aid, in which case they display or print the values of some or all of the object's data members before those members are deallocated. The name of a destructor is the class's name, preceded by a tilde (~).

Neither constructors nor destructors have return types, and neither use **return** statements. Both constructors and destructors can be explicitly called.

11.4.1.4 An Example

Our example of a C++ abstract data type is a stack:

```
#include <iostream.h>
class Stack {
    private:  /** These members are visible only to other
                /** members and friends (see Section 11.6.4)
        int *stackPtr;
        int maxLen;
        int topSub;
    public:   /** These members are visible to clients
        Stack() { /** A constructor
            stackPtr = new int [100];
            maxLen = 99;
            topSub = -1;
        }
        ~Stack() {delete [] stackPtr;}; /** A destructor
        void push(int number) {
            if (topSub == maxLen)
                cerr << "Error in push--stack is full\n";
            else stackPtr[++topSub] = number;
        }
        void pop() {
            if (empty())
                cerr << "Error in pop--stack is empty\n";
            else topSub--;
        }
        int top() {
```

```

        if (empty())
            cerr << "Error in top--stack is empty\n";
        else
            return (stackPtr[topSub]);
    }
    int empty() {return (topSub == -1);}
}

```

We discuss only a few aspects of this class definition, because it is not necessary to understand all of the details of the code. Objects of the `Stack` class are stack dynamic but include a pointer that references heap-dynamic data. The `Stack` class has three data members—`stackPtr`, `maxLen`, and `topSub`—all of which are private. `stackPtr` is used to reference the heap-dynamic data, which is the array that implements the stack. The class also has four public member functions—`push`, `pop`, `top`, and `empty`—as well as a constructor and a destructor. All of the member function definitions are included in this class, although they could have been externally defined. Because the bodies of the member functions are included, they are all implicitly inlined. The constructor uses the `new` operator to allocate an array of 100 `int` elements from the heap. It also initializes `maxLen` and `topSub`.

The following is an example program that uses the `Stack` abstract data type:

```

void main() {
    int topOne;
    Stack stk; /** Create an instance of the Stack class
    stk.push(42);
    stk.push(17);
    topOne = stk.top();
    stk.pop();
    . . .
}

```

Following is a definition of the `Stack` class with only prototypes of the member functions. This code is stored in a header file with the `.h` file name extension. The definitions of the member functions follow the class definition. These use the scope resolution operator, `::`, to indicate the class to which they belong. These definitions are stored in a code file with the file name extension `.cpp`.

```

// Stack.h - the header file for the Stack class
#include <iostream.h>
class Stack {
    private: /** These members are visible only to other
              /** members and friends (see Section 11.6.3)
        int *stackPtr;
        int maxLen;
        int topSub;
    public: /** These members are visible to clients
        Stack(); /** A constructor
        ~Stack(); /** A destructor
        void push(int);
        void pop();

```

```

        int top();
        int empty();
    }

// Stack.cpp - the implementation file for the Stack class
#include <iostream.h>
#include "Stack.h"
using std::cout;
Stack::Stack() { /** A constructor
    stackPtr = new int [100];
    maxLen = 99;
    topSub = -1;
}
Stack::~Stack() {delete [] stackPtr;}; /** A destructor
void Stack::push(int number) {
    if (topSub == maxLen)
        cerr << "Error in push--stack is full\n";
    else stackPtr[++topSub] = number;
}
void Stack::pop() {
    if (topSub == -1)
        cerr << "Error in pop--stack is empty\n";
    else topSub--;
}
int top() {
    if (topSub == -1)
        cerr << "Error in top--stack is empty\n";
    else
        return (stackPtr[topSub]);
}
int Stack::empty() {return (topSub == -1);}

```

11.4.2 Abstract Data Types in Java

Java support for abstract data types is similar to that of C++. There are, however, a few important differences. All objects are allocated from the heap and accessed through reference variables. Methods in Java must be defined completely in a class. A method body must appear with its corresponding method header.² Therefore, a Java abstract data type is both declared and defined in a single syntactic unit. A Java compiler can inline any method that is not overridden. Definitions are hidden from clients by declaring them to be private.

One important advantage of Java's classes over the classes of C++ is that it uses implicit garbage collection of all objects. This allows the programmer to ignore the issue of deallocation of objects and the clutter of deallocation code in the implementations of abstract data types.

2. Java interfaces are an exception to this—an interface has method headers but cannot include their bodies.

Rather than having private and public clauses in its class definitions, in Java access modifiers can be attached to method and variable definitions. If an instance variable or method does not have an access modifier, it has package access, which is discussed in Section 11.7.2.

11.4.2.1 An Example

The following is a Java class definition for our stack example:

```
class StackClass {
    private int [] stackRef;
    private int maxLen,
               topIndex;
    public StackClass() { // A constructor
        stackRef = new int [100];
        maxLen = 99;
        topIndex = -1;
    }
    public void push(int number) {
        if (topIndex == maxLen)
            System.out.println("Error in push-stack is full");
        else stackRef[++topIndex] = number;
    }
    public void pop() {
        if (empty())
            System.out.println("Error in pop-stack is empty");
        else --topIndex;
    }
    public int top() {
        if (empty()) {
            System.out.println("Error in top-stack is empty");
            return 9999;
        }
        else
            return (stackRef[topIndex]);
    }
    public boolean empty() {return (topIndex == -1);}
}
```

An example class that uses StackClass follows:

```
public class TstStack {
    public static void main(String[] args) {
        StackClass myStack = new StackClass();
        myStack.push(42);
        myStack.push(29);
        System.out.println("29 is: " + myStack.top());
        myStack.pop();
        System.out.println("42 is: " + myStack.top());
    }
}
```

```
        myStack.pop();  
        myStack.pop(); // Produces an error message  
    }  
}
```

One obvious difference between the Java and the C++ implementations of the stack is the lack of a destructor in the Java version, obviated by Java's implicit garbage collection.³

11.4.2.2 Evaluation

Although different in some primarily cosmetic ways, Java's support for abstract data types is similar to that of C++. Java clearly provides for what is necessary to design abstract data types.

11.4.3 Abstract Data Types in C#

Recall that C# is based on both C++ and Java and that it also includes some new constructs. Like Java, all C# class instances are heap dynamic. Default constructors, which provide initial values for instance data, are predefined for all classes. These constructors provide typical initial values, such as 0 for **int** types and **false** for **boolean** types. A user can furnish one or more constructors for any class he or she defines. Such constructors can assign initial values to some or all of the instance data of the class. Any instance variable that is not initialized in a user-defined constructor is assigned a value by the default constructor.

Although C# allows destructors to be defined, because it uses garbage collection for most of its heap objects, destructors are rarely used.

11.4.3.1 Encapsulation

C++ includes both classes and structs, which are nearly identical constructs. The only difference is that the default access modifier for class is **private**, whereas for structs it is **public**. C# also has structs, but they are very different from those of C++. In C#, structs are, in a sense, lightweight classes. They can have constructors, properties, methods, and data fields and can implement interfaces but do not support inheritance. One other important difference between structs and classes in C# is that structs are value types, as opposed to reference types. They are allocated on the run-time stack, rather than the heap. If they are passed as parameters, like other value types, by default they are passed by value. All C# value types, including all of its primitive types, are actually structs. Structs can be created by declaring them, like other predefined value types, such as **int** or **float**. They can also be created with the **new** operator, which calls a constructor to initialize them.

3. In Java, the **finalize** method serves as a kind of destructor.

Structs are used in C# primarily to implement relatively small simple types that need never be base types for inheritance. They are also used when it is convenient for the objects of the type to be stack as opposed to heap allocated.

11.4.3.2 Information Hiding

C# uses the **private** and **protected** access modifiers exactly as they are used in Java.

C# provides properties, which it inherited from Delphi, as a way of implementing getters and setters without requiring explicit method calls by the client. Properties provide implicit access to specific private instance data. For example, consider the following simple class and client code:

```
public class Weather {
    public int DegreeDays { /** DegreeDays is a property
        get {
            return degreeDays;
        }
        set {
            if(value < 0 || value > 30)
                Console.WriteLine(
                    "Value is out of range: {0}", value);
            else
                degreeDays = value;
        }
    }
    private int degreeDays;
    . . .
}
. . .
Weather w = new Weather();
int degreeDaysToday, oldDegreeDays;
. . .
w.DegreeDays = degreeDaysToday;
. . .
oldDegreeDays = w.DegreeDays;
```

In the class `Weather`, the property `DegreeDays` is defined. This property provides a getter method and a setter method for access to the private data member, `degreeDays`. In the client code following the class definition, `degreeDays` is treated as if it were a public-member variable, although access to it is available through the property only. Notice the use of the implicit variable **value** in the setter method. This is the mechanism by which the new value of the property is referenced.

The stack example is not shown here in C#. The only difference between the Java version in Section 11.4.2.1 and the C# version is the output method calls and the use of **bool** instead of **boolean** for the return type of the `empty` method.

11.4.4 Abstract Data Types in Ruby

Ruby provides support for abstract data types through its classes. In terms of capabilities, Ruby classes are similar to those in C++ and Java.

11.4.4.1 Encapsulation

In Ruby, a class is defined in a compound statement opened with the **class** reserved word and closed with **end**. The names of instance variables have a special syntactic form, they must begin with at signs (@). Instance methods have the same syntax as functions in Ruby: They begin with the **def** reserved word and end with **end**. Class methods are distinguished from instance methods by having the class name appended to the beginning of their names with a period separator. For example, in a class named `Stack`, a class method's name would begin with `Stack`. Constructors in Ruby are named `initialize`. Because the constructor cannot be overloaded, there only can be one per class.

Classes in Ruby are dynamic in the sense that members can be added at any time. This is done by simply including additional class definitions that specify the new members. Moreover, even predefined classes of the language, such as `String`, can be extended. For example, consider the following class definition:

```
class myClass
  def meth1
    . . .
  end
end
```

This class could be extended by adding a second method, `meth2`, with a second class definition:

```
class myClass
  def meth2
    . . .
  end
end
```

Methods can also be removed from a class. This is done by providing another class definition in which the method to be removed is sent to the method `remove_method` as a parameter. The dynamic classes of Ruby is another example of a language designer trading readability (and as a consequence, reliability) for flexibility. Allowing dynamic changes to classes clearly adds flexibility to the language, while harming readability. To determine the behavior of a class at a particular point in a program, one must find all of its definitions in the program and consider all of them.

11.4.4.2 Information Hiding

Access control for methods in Ruby is dynamic, so access violations are detected only during execution. The default method access is public, but it can also be protected or private. There are two ways to specify the access control, both of which use functions with the same names as the access levels, **private** and **public**. One way is to call the appropriate function without parameters. This resets the default access for subsequently defined methods in the class. For example,

```
class MyClass
  def meth1
    . . .
  end
  . . .
private
  def meth7
    . . .
  end
  . . .
end # of class MyClass
```

The alternative is to call the access control functions with the names of the specific methods as parameters. For example, the following is semantically equivalent to the previous class definition:

```
class MyClass
  def meth1
    . . .
  end
  . . .
  def meth7
    . . .
  end
  private :meth7, . . .
end # of class MyClass
```

In Ruby, all data members of a class are private, and that cannot be changed. So, data members can be accessed only by the methods of the class, some of which may be accessor methods. In Ruby, instance data that are accessible through accessor methods are called **attributes**.

For an instance variable named `@sum`, the getter and setter methods would be as follows:

```
def sum
  @sum
end
def sum=(new_sum)
  @sum = new_sum
end
```

Notice that getters are given the name of the instance variable minus the @. The names of setter methods are the same as those of the corresponding getters, except they have an equal sign (=) attached.

Getters and setters can be implicitly generated by the Ruby system by including calls to `attr_reader` and `attr_writer`, respectively, in the class definition. The parameters to these are the symbols of the attribute's names, as is illustrated in the following:

```
attr_reader :sum, :total
attr_writer :sum
```

11.4.4.3 An Example

Following is the stack example written in Ruby:

```
# Stack.rb - defines and tests a stack of maximum length
#             100, implemented in an array
class StackClass

# Constructor
  def initialize
    @stackRef = Array.new(100)
    @maxLen = 100
    @topIndex = -1
  end

# push method
  def push(number)
    if @topIndex == @maxLen
      puts "Error in push - stack is full"
    else
      @topIndex = @topIndex + 1
      @stackRef[@topIndex] = number
    end
  end

# pop method
  def pop
    if empty
      puts "Error in pop - stack is empty"
    else
      @topIndex = @topIndex - 1
    end
  end

# top method
  def top
    if empty
      puts "Error in top - stack is empty"
    else
      @stackRef[@topIndex]
    end
  end
end
```

```

# empty method
def empty
  @topIndex == -1
end
end # of Stack class

# Test code for StackClass
myStack = StackClass.new
myStack.push(42)
myStack.push(29)
puts "Top element is (should be 29): #{myStack.top}"
myStack.pop
puts "Top element is (should be 42): #{myStack.top}"
myStack.pop
# The following pop should produce an
# error message - stack is empty
myStack.pop

```

Recall that the notation `#{variable}` converts the value of the variable to a string, which is then inserted into the string in which it appears. This class defines a stack structure that can store objects of any type.

11.4.4.4 Evaluation

Recall that in Ruby, everything is an object and arrays are actually arrays of references to objects. That clearly makes this stack more flexible than the similar examples in C++ and Java. Furthermore, simply by passing the desired maximum length to the constructor, objects of this class could have any given maximum length. Of course, because arrays in Ruby have dynamic length, the class could be modified to implement stack objects that are not restricted to any length, except that imposed by the machine's memory capacity. Because the names of class and instance variables have different forms, Ruby has a slight readability advantage over the other languages discussed in this section.

11.5 Parameterized Abstract Data Types

It is often convenient to be able to parameterize abstract data types. For example, we should be able to design a stack abstract data type that can store any scalar type elements rather than be required to write a separate stack abstraction for every different scalar type. Note that this is only an issue for static typed languages. In a dynamic typed language like Ruby, any stack implicitly can store any type elements. In fact, different elements of the stack could be of different types. In the following three subsections, the capabilities of C++, Java 5.0, and C# 2005 to construct parameterized abstract data types are discussed.

11.5.1 C++

To make the example C++ stack class of Section 11.4.1 generic in the stack size, only the constructor function needs to be changed, as in the following:

```
Stack(int size) {
    stackPtr = new int [size];
    maxLen = size - 1;
    topSub = -1;
}
```

The declaration for a stack object now may appear as follows:

```
Stack stk(150);
```

The class definition for `Stack` can include both constructors, so users can use the default-size stack or specify some other size.

The element type of the stack can be made generic by making the class a templated class. Then, the element type can be a template parameter. The definition of the templated class for a stack type is as follows:

```
#include <iostream.h>
template <typename Type> // Type is the template parameter
class Stack {
    private:
        Type *stackPtr;
        int maxLen;
        int topSub;
    public:
        // A constructor for 100 element stacks
        Stack() {
            stackPtr = new Type [100];
            maxLen = 99;
            topSub = -1;
        }
        // A constructor for a given number of elements
        Stack(int size) {
            stackPtr = new Type [size];
            maxLen = size - 1;
            topSub = -1;
        }
        ~Stack() {delete stackPtr;}; // A destructor
        void push(Type number) {
            if (topSub == maxLen)
                cout << "Error in push-stack is full\n";
            else stackPtr[++ topSub] = number;
        }
        void pop() {
```

```

        if (empty())
            cout << "Error in pop-stack is empty\n";
        else topSub --;
    }
    Type top() {
        if (empty())
            cerr << "Error in top--stack is empty\n";
        else
            return (stackPtr[topSub]);
    }
    int empty() {return (topSub == -1);}
}

```

C++ templated classes are instantiated to become typed classes at compile time. For example, an instance of the templated `Stack` class, as well as an instance of the typed class, can be created with the following declaration:

```
Stack<int> myIntStack;
```

However, if an instance of the templated `Stack` class has already been created for the `int` type, the typed class need not be created.

11.5.2 Java 5.0

Java 5.0 supports a form of parameterized abstract data types in which the generic parameters must be classes. Recall that these were briefly discussed in Chapter 9.

The most common generic types are collection types, such as `LinkedList` and `ArrayList`, which were in the Java class library before support for generics was added. The original collection types stored `Object` class instances, so they could store any objects (but not primitive types). Therefore, the collection types have always been able to store multiple types (as long as they are classes). There were three issues with this: First, every time an object was removed from the collection, it had to be cast to the appropriate type. Second, there was no error checking when elements were added to the collection. This meant that once the collection was created, objects of any class could be added to the collection, even if the collection was meant to store only `Integer` objects. Third, the collection types could not store primitive types. So, to store `int` values in an `ArrayList`, the value first had to be put in an `Integer` class instance. For example, consider the following code:

```

/* Create an ArrayList object
ArrayList myArray = new ArrayList();
/* Create an element
myArray.add(0, new Integer(47));
/* Get first object
Integer myInt = (Integer)myArray.get(0);

```

In Java 5.0, the collection classes, the most commonly used of which is `ArrayList`, became a generic class. Such classes are instantiated by calling **new** on the class constructor and passing it the generic parameter in pointed brackets. For example, the `ArrayList` class can be instantiated to store `Integer` objects with the following statement:

```
ArrayList <Integer> myArray = new ArrayList <Integer>();
```

This new class overcomes two of the problems with pre-Java 5.0 collections. Only `Integer` objects can be put into the `myArray` collection. Furthermore, there is no need to cast an object being removed from the collection.

Java 5.0 also includes generic collections for linked-lists, queues, and sets.

Users also can define generic classes in Java 5.0. For example, we could have the following:

```
public class MyClass<T> {
    . . .
}
```

This class could be instantiated with the following:

```
MyClass<String> myString;
```

There are some drawbacks to these user-defined generic classes. For one thing, they cannot store primitives. Second, the elements cannot be indexed. Elements must be added to user-defined generic collections with the `add` method. Next, we implement the generic stack example using an `ArrayList`. Note that the last element of an `ArrayList` is found using the `size` method, which returns the number of elements in the structure. Elements are deleted from the structure with the `remove` method. Following is the generic class:

```
import java.util.*;
public class Stack2<T> {

    private ArrayList<T> stackRef;
    private int maxLen;
    public Stack2() { // A constructor
        stackRef = new ArrayList<T> ();
        maxLen = 99;
    }
    public void push(T newValue) {
        if (stackRef.size() == maxLen)
            System.out.println("Error in push-stack is full");
        else
            stackRef.add(newValue);
    }
    public void pop() {
        if (empty())
```



```

        System.out.println("Error in pop-stack is empty");
    else
        stackRef.remove(stackRef.size() - 1);
    }
    public T top() {
        if empty() {
            System.out.println("Error in top-stack is empty");
            return null; }
        else
            return (stackRef.get(stackRef.size() - 1));
    }
    public boolean empty() {return (stackRef.isEmpty());}

```

This class could be instantiated for the `String` type with the following:

```
Stack2<String> myStack = new Stack2<String>();
```

Recall from Chapter 9, that Java 5.0 supports wildcard classes. For example, `Collection<?>` is a wildcard class for all collection classes. This allows a method to be written that can accept any collection type as a parameter. Because a collection can itself be generic, the `Collection<?>` class is in a sense a generic of a generic class.

Some care must be taken with objects of the wildcard type. For example, because the components of a particular object of this type have a type, other type objects cannot be added to the collection. For example, consider

```
Collection<?> c = new ArrayList<String>();
```

It would be illegal to use the `add` method to put something into this collection unless its type were `String`.

A generic class can easily be defined in Java 5.0 that will work only for a restricted set of types. For example, a class can declare a variable of the generic type and call a method such as `compareTo` through that variable. If the class is instantiated for a type that does not include a `compareTo` method, the class cannot be used. To prevent a generic class from being instantiated for a type that does not support `compareTo`, it could be defined with the following generic parameter:

```
<T extends Comparable>
```

`Comparable` is the interface in which `compareTo` is declared. If this generic type is used on a class definition, the class cannot be instantiated for any type that does not implement `Comparable`. The choice of the reserved word **extends** seems odd here, but its use is related to the concept of a subtype. Apparently, the designers of Java did not want to add another more connotative reserved word to the language.

11.5.3 C# 2005

As was the case with Java, the first version of C# defined collection classes that stored objects of any class. These were `ArrayList`, `Stack`, and `Queue`. These classes had the same problems as the collection classes of pre-Java 5.0.

Generic classes were added to C# in its 2005 version. The five predefined generic collections are `Array`, `List`, `Stack`, `Queue`, and `Dictionary` (the `Dictionary` class implements hashes). Exactly as in Java 5.0, these classes eliminate the problems of allowing mixed types in collections and requiring casts when objects are removed from the collections.

As with Java 5.0, users can define generic classes in C# 2005. One capability of the user-defined C# generic collections is that any of them can be defined to allow its elements to be indexed (accessed through subscripting). Although the indexes are usually integers, an alternative is to use strings as indexes.

One capability that Java 5.0 provides that C# 2005 does not is wildcard classes.

11.6 Encapsulation Constructs

The first five sections of this chapter discussed abstract data types, which are minimal encapsulations. This section describes the multiple-type encapsulations that are needed for larger programs.

11.6.1 Introduction

When the size of a program reaches beyond a few thousand lines, two practical problems become evident. From the programmer's point of view, having such a program appear as a single collection of subprograms or abstract data type definitions does not impose an adequate level of organization on the program to keep it intellectually manageable. The second practical problem for larger programs is recompilation. For relatively small programs, recompiling the whole program after each modification is not costly. But for large programs, the cost of recompilation is significant. So, there is an obvious need to find ways to avoid recompilation of the parts of a program that are not affected by a change. The obvious solution to both of these problems is to organize programs into collections of logically related code and data, each of which can be compiled without recompilation of the rest of the program. An **encapsulation** is such a collection.

Encapsulations are often placed in libraries and made available for reuse in programs other than those for which they were written. People have been writing programs with more than a few thousand lines for at least the last 50 years, so techniques for providing encapsulations have been evolving for some time.

In languages that allow nested subprograms, programs can be organized by nesting subprogram definitions inside the logically larger subprograms that

use them. This can be done in Python and Ruby. As discussed in Chapter 5, however, this method of organizing programs, which uses static scoping, is far from ideal. Therefore, even in languages that allow nested subprograms, they are not used as a primary organizing encapsulation construct.

11.6.2 Encapsulation in C

C does not provide complete support for abstract data types, although both abstract data types and multiple-type encapsulations can be simulated.

In C, a collection of related functions and data definitions can be placed in a file, which can be independently compiled. Such a file, which acts as a library, has an implementation of its entities. The interface to such a file, including data, type, and function declarations, is placed in a separate file called a **header file**. Type representations can be hidden by declaring them in the header file as pointers to struct types. The complete definitions of such struct types need only appear in the implementation file.

The header file, in source form, and the compiled version of the implementation file are furnished to clients. When such a library is used, the header file is included in the client code, using an `#include` preprocessor specification, so that references to functions and data in the client code can be type checked. The `#include` specification also documents the fact that the client program depends on the library implementation file. This approach effectively separates the specification and implementation of an encapsulation.

Although these encapsulations work, they create some insecurities. For example, a user could simply cut and paste the definitions from the header file into the client program, rather than using `#include`. This would work, because `#include` simply copies the contents of its operand file into the file in which the `#include` appears. However, there are two problems with this approach. First, the documentation of the dependence of the client program on the library (and its header file) is lost. Second, suppose a user copies the header file into his or her program. Then the author of the library changes both the header and the implementation files. Following this, the user uses the new implementation file with the old header. For example, a variable `x` could have been defined to be `int` type in the old header file, which the client code still uses, although the implementation code has been recompiled with the new header file, which defines `x` to be `float`. So, the implementation code was compiled with `x` as an `int` but the client code was compiled with `x` as a `float`. The linker does not detect this error.

Thus, it is the user's responsibility to ensure that both the header and implementation files are up-to-date. This is often done with a `make` utility.

One other drawback of this approach is the inherent problems of pointers and the potential confusion with assignment and comparisons of pointers.

11.6.3 Encapsulation in C++

C++ provides two different kinds of encapsulation: header and implementation files can be defined as in C, or class headers and definitions can be defined. Because of the complex interplay of C++ templates and separate compilation,

the header files of C++ template libraries often include complete definitions of resources, rather than just data declarations and subprogram protocols; this is due in part to the use of the C linker for C++ programs.

When nontemplated classes are used for encapsulations, the class header file has only the prototypes of the member functions, with the function definitions provided outside the class in a code file, as in the last example in Section 11.4.1.4. This clearly separates interface from implementation.

One language design problem that results from having classes but no generalized encapsulation construct is that sometimes when operations are defined that use two different classes of objects, the operation does not naturally belong in either class. For example, suppose we have an abstract data type for matrices and one for vectors and need a multiplication operation between a vector and a matrix. The multiplication code must have access to the data members of both the vector and the matrix classes, but neither of those classes is the natural home for the code. Furthermore, regardless of which is chosen, access to the members of the other is a problem. In C++, these kinds of situations can be handled by allowing nonmember functions to be “friends” of a class. Friend functions have access to the private entities of the class where they are declared to be friends. For the matrix/vector multiplication operation, one C++ solution is to define the operation outside both the matrix and the vector classes but define it to be a friend of both. The following skeletal code illustrates this scenario:

```
class Matrix; /** A class declaration
class Vector {
    friend Vector multiply(const Matrix&, const Vector&);
    . . .
};
class Matrix { /** The class definition
    friend Vector multiply(const Matrix&, const Vector&);
    . . .
};
/** The function that uses both Matrix and Vector objects
Vector multiply(const Matrix& m1, const Vector& v1) {
    . . .
}
```

In addition to functions, whole classes can be defined to be friends of a class; then all the private members of the class are visible to all of the members of the friend class.

11.6.4 C# Assemblies

C# includes a larger encapsulation construct than a class. The construct is the one used by all of the .NET programming languages: the assembly. Assemblies are built by .NET compilers. A .NET application consists of one or more assemblies. An **assembly** is a file⁴ that appears to application programs to be a

4. An assembly can consist of any number of files.

single dynamic link library (.dll)⁵ or an executable (.exe). An assembly defines a module, which can be separately developed. An assembly includes several different components. One of the primary components of an assembly is its programming code, which is in an intermediate language, having been compiled from its source language. In .NET, the intermediate language is named Common Intermediate Language (CIL). It is used by all .NET languages. Because its code is in CIL, an assembly can be used on any architecture, device, or operating system. When executed, the CIL is just-in-time compiled to native code for the architecture on which it is resident.

In addition to the CIL code, a .NET assembly includes metadata that describes every class it defines, as well as all external classes it uses. An assembly also includes a list of all assemblies referenced in the assembly and an assembly version number.

In the .NET world, the assembly is the basic unit of deployment of software. Assemblies can be private, in which case they are available to just one application, or public, which means any application can use them.

As mentioned previously, C# has an access modifier, **internal**. An **internal** member of a class is visible to all classes in the assembly in which it appears.

Java has a file structure that is similar to an assembly called a Java Archive (JAR). It is also used for deployment of Java software systems. JARs are built with the Java utility `jar`, rather than a compiler.

11.7 Naming Encapsulations

We have considered encapsulations to be syntactic containers for logically related software resources—in particular, abstract data types. The purpose of these encapsulations is to provide a way to organize programs into logical units for compilation. This allows parts of programs to be recompiled after isolated changes. There is another kind of encapsulation that is necessary for constructing large programs: a naming encapsulation.

A large program is usually written by many developers, working somewhat independently, perhaps even in different geographic locations. This requires the logical units of the program to be independent, while still able to work together. It also creates a naming problem: How can independently working developers create names for their variables, methods, and classes without accidentally using names already in use by some other programmer developing a different part of the same software system?

5. A **dynamic link library (DLL)** is a collection of classes and methods that are individually linked to an executing program when needed during execution. Therefore, although a program has access to all of the resources in a particular DLL, only the parts that are actually used are ever loaded and linked to the program. DLLs have been part of the Windows programming environment since Windows first appeared. However, the DLLs of .NET are quite different from those of previous Windows systems.

Libraries are the origin of the same kind of naming problems. Over the past two decades, large software systems have become progressively more dependent on libraries of supporting software. Nearly all software written in contemporary programming languages requires the use of large and complex standard libraries, in addition to application-specific libraries. This widespread use of multiple libraries has necessitated new mechanisms for managing names. For example, when a developer adds new names to an existing library or creates a new library, he or she must not use a new name that conflicts with a name already defined in a client's application program or in some other library the program uses. Without some language processor assistance, this is virtually impossible, because there is no convenient way for the library author to know what names a client's program uses or what names are defined by the other libraries the client program might use.

Naming encapsulations define name scopes that assist in avoiding these name conflicts. Each library can create its own naming encapsulation to prevent its names from conflicting with the names defined in other libraries or in client code. Each logical part of a software system can create a naming encapsulation with the same purpose.

Naming encapsulations are logical encapsulations, in the sense that they need not be physically contiguous. Several different collections of code can be placed in the same namespace, even though they are stored in different places. In the following sections, we briefly describe the uses of naming encapsulations in C++, Java, and Ruby.

11.7.1 C++ Namespaces

C++ includes a specification, **namespace**, that helps programs manage the problem of global namespaces. One can place each library in its own namespace and qualify the names in the program with the name of the namespace when the names are used outside that namespace. For example, suppose there is an abstract data type header file that implements stacks. If there is concern that some other library file may define a name that is used in the stack abstract data type, the file that defines the stack could be placed in its own namespace. This is done by placing all of the declarations for the stack in a namespace block, as in the following:

```
namespace myStackSpace {  
    // Stack declarations  
}
```

The implementation file for the stack abstract data type could reference the names declared in the header file with the scope resolution operator, `::`, as in

```
myStackSpace::topSub
```

The implementation file could also appear in a namespace block specification identical to the one used on the header file, which would make all of

the names declared in the header file directly visible. This is definitely simpler, but slightly less readable, because it is less obvious where a specific name in the implementation file is declared.

Client code can gain access to the names in the namespace of the header file of a library in three different ways. One way is to qualify the names from the library with the name of the namespace. For example, a reference to the variable `topSub` could appear as follows:

```
myStackSpace::topSub
```

This is exactly the way the implementation code could reference it if the implementation file was not in the same namespace.

The other two approaches use the **using** directive. This directive can be used to qualify individual names from a namespace, as with

```
using myStackSpace::topSub;
```

which makes `topSub` visible, but not any other names from the `myStackSpace` namespace.

The **using** directive also can be used to qualify all of the names from a namespace, as in the following:

```
using namespace myStackSpace;
```

Code that includes this directive can directly access the names defined in the namespace, as in

```
p = topSub;
```

Be aware that namespaces are a complicated feature of C++, and we have introduced only the simplest part of the story here.

C# includes namespaces that are much like those of C++.

11.7.2 Java Packages

Java includes a naming encapsulation construct: the package. Packages can contain more than one type⁶ definition, and the types in a package are partial friends of one another. *Partial* here means that the entities defined in a type in a package that either are public or protected (see Chapter 12) or have no access specifier are visible to all other types in the package.

Entities without access modifiers are said to have **package scope**, because they are visible throughout the package. Java therefore has less need for explicit

6. By type here we mean either a class or an interface.

friend declarations and does not include the friend functions or friend classes of C++.

The resources defined in a file are specified to be in a particular package with a package declaration, as in

```
package stkpkg;
```

The package declaration must appear as the first line of the file. The resources of every file that does not include a package declaration are implicitly placed in the same unnamed package.

The clients of a package can reference the types defined in the package using fully qualified names. For example, if the package `stkpkg` has a class named `myStack`, that class can be referenced in a client of `stkpkg` as `stkpkg.myStack`. Likewise, a variable in the `myStack` object named `topSub` could be referenced as `stkpkg.myStack.topSub`. Because this approach can quickly become cumbersome when packages are nested, Java provides the **import** declaration, which allows shorter references to type names defined in a package. For example, suppose the client includes the following:

```
import stkpkg.myStack;
```

Now, the class `myStack` can be referenced by just its name. To be able to access all of the type names in the package, an asterisk can be used on the import statement in place of the type name. For example, if we wanted to import all of the types in `stkpkg`, we could use the following:

```
import stkpkg.*;
```

Note that Java's **import** is only an abbreviation mechanism. No otherwise hidden external resources are made available with **import**. In fact, in Java nothing is implicitly hidden if it can be found by the compiler or class loader (using the package name and the `CLASSPATH` environment variable).

Java's **import** documents the dependencies of the package in which it appears on the packages named in the **import**. These dependencies are less obvious when **import** is not used.

11.7.3 Ruby Modules

Ruby classes serve as namespace encapsulations, as do the classes of other languages that support object-oriented programming. Ruby has an additional naming encapsulation, called a **module**. Modules typically define collections of methods and constants. So, modules are convenient for encapsulating libraries of related methods and constants, whose names are in a separate namespace so there are no name conflicts with other names in a program that uses the module. Modules are unlike classes in that they cannot be instantiated or subclassed and do not define variables. Methods that are defined in a module include the

module's name in their names. For example, consider the following skeletal module definition:

```

module MyStuff
  PI = 3.14159265
  def MyStuff.mymethod1 (p1)
    . . .
  end
  def MyStuff.mymethod2 (p2)
    . . .
  end
end

```

Assuming the `MyStuff` module is stored in its own file, a program that wants to use the constant and methods of `MyStuff` must first gain access to the module. This is done with the `require` method, which takes the file name in the form of a string literal as a parameter. Then, the constants and methods of the module can be accessed through the module's name. Consider the following code that uses our example module, `MyStuff`, which is stored in the file named `myStuffMod`:

```

require 'myStuffMod'
. . .
MyStuff.mymethod1 (x)
. . .

```

Modules are further discussed in Chapter 12.

S U M M A R Y

The concept of abstract data types, and their use in program design, was a milestone in the development of programming as an engineering discipline. Although the concept is relatively simple, its use did not become convenient and safe until languages were designed to support it.

The two primary features of abstract data types are the packaging of data objects with their associated operations and information hiding. A language may support abstract data types directly or simulate them with more general encapsulations.

C++ data abstraction is provided by classes. Classes are types, and instances can be either stack or heap dynamic. A member function (method) can have its complete definition appear in the class or have only the protocol given in the class and the definition placed in another file, which can be separately compiled. C++ classes can have two clauses, each prefixed with an access modifier: `private` or `public`. Both constructors and destructors can be given in class definitions. Heap-allocated objects must be explicitly deallocated with **delete**.

Java data abstractions are similar to those of C++, except all Java objects are allocated from the heap and are accessed through reference variables. Also, all objects are garbage collected. Rather than having access modifiers attached to clauses, in Java the modifiers appear on individual declarations (or definitions).

C# supports abstract data types with both classes and structs. Its structs are value types and do not support inheritance. C# classes are similar to those of Java.

Ruby supports abstract data types with its classes. Ruby's classes differ from those of most other languages in that they are dynamic—members can be added, deleted, or changed during execution.

C++, Java 5.0, and C# 2005 allow their abstract data types to be parameterized—Ada through its generic packages, C++ through its templated classes, and Java 5.0 and C# through their collection classes and interfaces and user-defined generic classes.

To support the construction of large programs, some contemporary languages include multiple-type encapsulation constructs, which can contain a collection of logically related types. An encapsulation may also provide access control to its entities. Encapsulations provide the programmer with a method of organizing programs that also facilitates recompilation.

C++, C#, Java, and Ruby provide naming encapsulations. For Ada and Java, they are named packages; for C++ and C#, they are namespaces; for Ruby, they are modules. Partially because of the availability of packages, Java does not have friend functions or friend classes.

REVIEW QUESTIONS

1. What are the two kinds of abstractions in programming languages?
2. Define *abstract data type*.
3. What are the advantages of the two parts of the definition of *abstract data type*?
4. What are the language design requirements for a language that supports abstract data types?
5. What are the language design issues for abstract data types?
6. From where are C++ objects allocated?
7. In what different places can the definition of a C++ member function appear?
8. What is the purpose of a C++ constructor?
9. What are the legal return types of a constructor?
10. Where are all Java methods defined?
11. How are C++ class instances created?
12. From where are Java class instances allocated?
13. Why does Java not have destructors?

14. Where are all Java methods defined?
15. Where are Java classes allocated?
16. Why are destructors not as frequently needed in Java as they are in C++?
17. What is a friend function? What is a friend class?
18. What is one reason Java does not have friend functions or friend classes?
19. Describe the fundamental differences between C# structs and its classes.
20. How is a struct object in C# created?
21. Explain the three reasons accessors to private types are better than making the types public.
22. What are the differences between a C++ struct and a C# struct?
23. What is the name of all Ruby constructors?
24. What is the fundamental difference between the classes of Ruby and those of C++ and Java?
25. How are instances of C++ template classes created?
26. Describe the two problems that appear in the construction of large programs that led to the development of encapsulation constructs.
27. What problems can occur using C to define abstract data types?
28. What is a C++ namespace, and what is its purpose?
29. What is a Java package, and what is its purpose?
30. Describe a .NET assembly.
31. What elements can appear in a Ruby module?

PROBLEM SET

1. Some software engineers believe that all imported entities should be qualified by the name of the exporting program unit. Do you agree? Support your answer.
2. Suppose someone designed a stack abstract data type in which the function `top` returned an access path (or pointer) rather than returning a copy of the top element. This is not a true data abstraction. Why? Give an example that illustrates the problem.
3. Write an analysis of the similarities of and differences between Java packages and C++ namespaces.
4. Discuss the advantages of C# properties, relative to writing accessor methods in C++ or Java.
5. Explain the dangers of C's approach to encapsulation.
6. Why didn't C++ eliminate the problems discussed in Problem 5?
7. Why are destructors rarely used in Java but essential in C++?

8. What are the arguments for and against the C++ policy on inlining of methods?
9. Describe a situation where a C# struct is preferable to a C# class.
10. Explain why naming encapsulations are important for developing large programs.
11. Describe the three ways a client can reference a name from a namespace in C++.
12. The namespace of the C# standard library, `System`, is not implicitly available to C# programs. Do you think this is a good idea? Defend your answer.
13. What are the advantages and disadvantages of the ability to change objects in Ruby?
14. Compare Java's packages with Ruby's modules.

PROGRAMMING EXERCISES

1. Design an abstract data type for a matrix with integer elements in a language that you know, including operations for addition, subtraction, and matrix multiplication.
2. Design a queue abstract data type for float elements in a language that you know, including operations for enqueue, dequeue, and empty. The dequeue operation removes the element and returns its value.
3. Modify the C++ class for the abstract stack type shown in Section 11.4.1 to use a linked list representation and test it with the same code that appears in this chapter.
4. Modify the Java class for the abstract stack type shown in Section 11.4.2 to use a linked list representation and test it with the same code that appears in this chapter.
5. Write an abstract data type for complex numbers, including operations for addition, subtraction, multiplication, division, extraction of each of the parts of a complex number, and construction of a complex number from two floating-point constants, variables, or expressions. Use C++, Java, C#, or Ruby.
6. Write an abstract data type for queues whose elements store 10-character names. The queue elements must be dynamically allocated from the heap. Queue operations are enqueue, dequeue, and empty. Use either C++, Java, C#, or Ruby.
7. Write an abstract data type for a queue whose elements can be any primitive type. Use Java 5.0, C# 2005, or C++.
8. Write an abstract data type for a queue whose elements include both a 20-character string and an integer priority. This queue must have the

following methods: enqueue, which takes a string and an integer as parameters; dequeue, which returns the string from the queue that has the highest priority; and empty. The queue is not to be maintained in priority order of its elements, so the dequeue operation must always search the whole queue.

9. A deque is a double-ended queue, with operations adding and removing elements from either end. Modify the solution to Programming Exercise 7 to implement a deque.
10. Write an abstract data type for rational numbers (a numerator and a denominator). Include a constructor and methods for getting the numerator, getting the denominator, addition, subtraction, multiplication, division, equality testing, and display. Use Java, C#, C++, or Ruby.